0684. Redundant Connection

edium | Category: Graphs | LeetCode: https://leetcode.com/problems/redundant-

Problem

- A tree of n nodes had one extra edge added. Return the edge that can be removed to restore a tree (the last such edge in input order).

Intuition

- Process edges in order with Union-Find. The first edge whose endpoints are already connected closes a cycle: that is the answer.

Brute Force

- For each edge, remove it and test connectivity/acyclicity with DFS: O(E (V + E)).

Optimal Approach

- Parent[i] = i, rank.
- For edge (u, v): if find(u) == find(v) return edge; else union.
- Path compression + union by rank make each operation nearly O(1).

Worked Example

```
[[1,2],[1,3],[2,3]].

[1,2] union -> {1,2}
[1,3] union -> {1,2,3}
[2,3] find(2) == find(3) -> redundant [2,3]
```

Java Solution

```java
public class RedundantConnection {
    private int[] parent, rank;

    public int[] findRedundantConnection(int[][] edges) {
        int n = edges.length;
        parent = new int[n + 1];
        rank = new int[n + 1];
        for (int i = 0; i <= n; i++) parent[i] = i;
        for (int[] e : edges) if (!union(e[0], e[1])) return e;
        return new int[0];
    }

    private int find(int x) {
        while (parent[x] != x) { parent[x] = parent[parent[x]]; x = parent[x]; }
        return x;
    }

    private boolean union(int a, int b) {
        int ra = find(a), rb = find(b);
        if (ra == rb) return false;
        if (rank[ra] < rank[rb]) { int t = ra; ra = rb; rb = t; }
        parent[rb] = ra;
        if (rank[ra] == rank[rb]) rank[ra]++;
        return true;
    }
}
```

Complexity

- Time: O(E alpha(V)). Space: O(V).

Pitfalls

- Nodes are 1-indexed; size the parent array n + 1.